Lec3

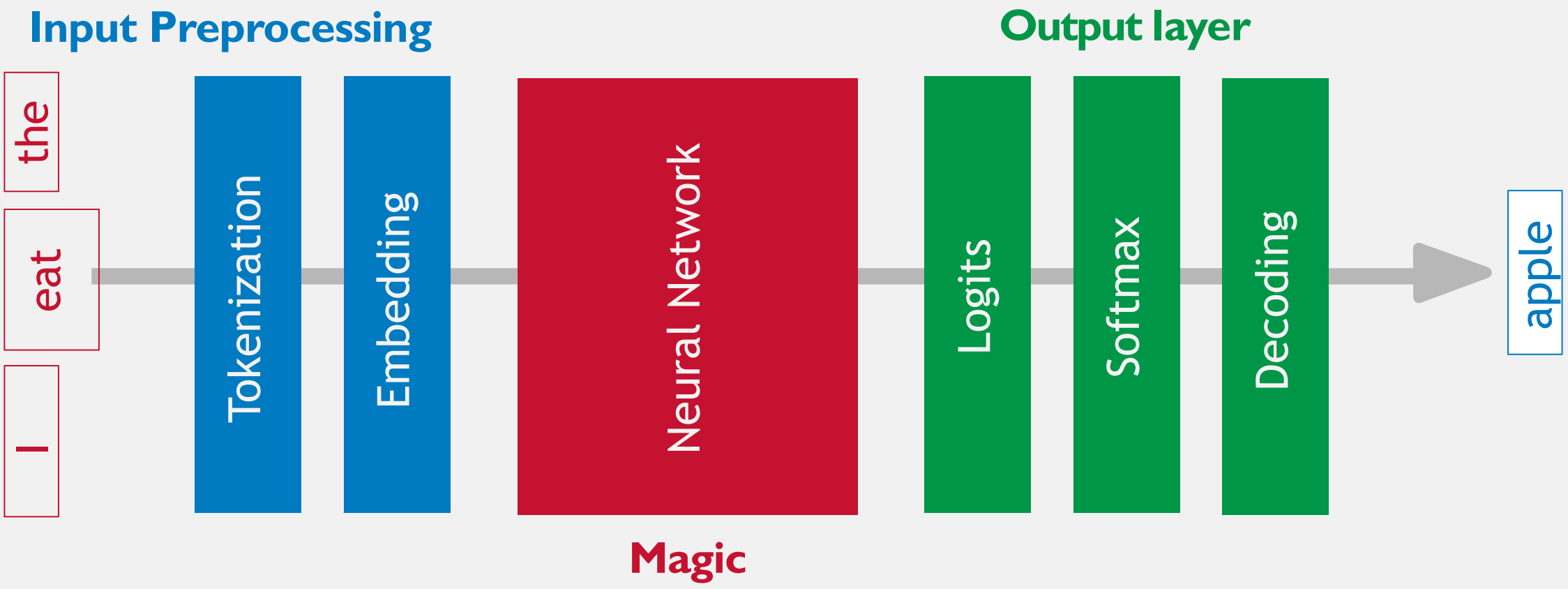
Meiying Zhang

COE201 - AI Large Models

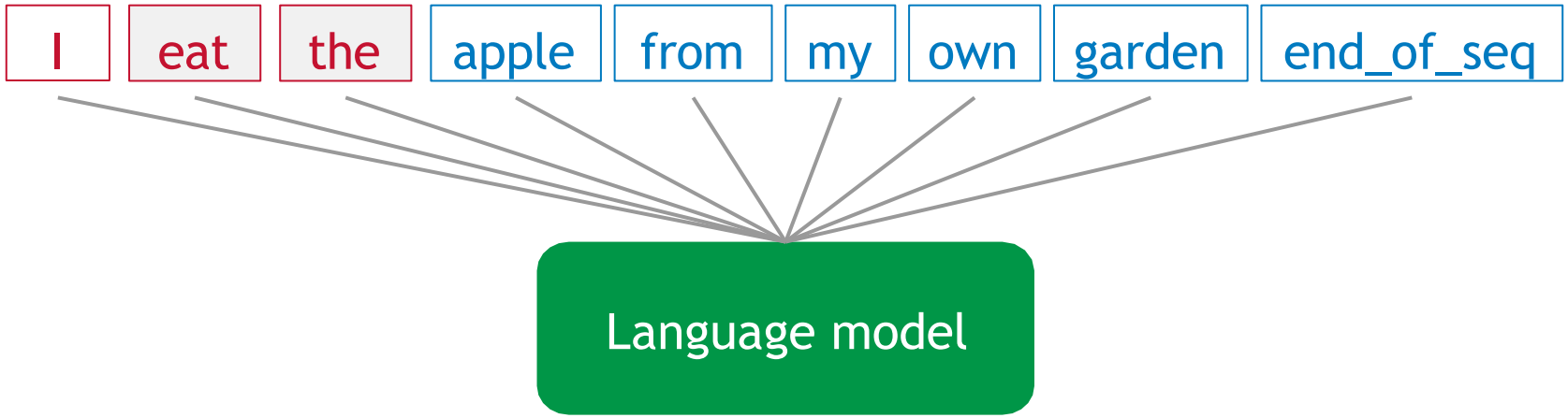
- Lectures: Monday 10:20-12:10AM
- Software labs: Monday 4:20-6:10PM
- Office hours: Thursday 2:30-3:30PM
- For personal /sensitive matters: zhangmy@sustech.edu.cn

Agenda

1. Input preprocessing
   - One-hot vector
   - Word2vec
   - Tokenization
   - Embedding


Input Preprocessing
I
eat
the
Tokenization
Embedding
Neural Network
Magic
Logits
Softmax
Decoding
Output layer
apple

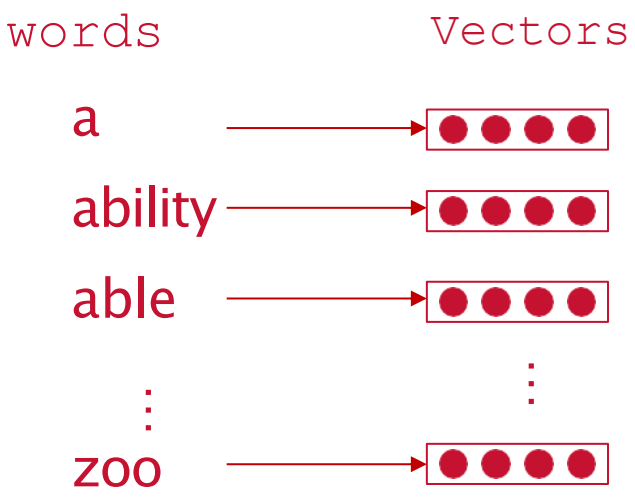
Language Model



Input Preprocessing

Input representation

Neural networks do not operate on discrete words.



2001

One-hot vector
- NNLM

2013

Word2Vec
- GloVe

2018

Tokenization
- ELMO
- Bert
- GPT

One-hot representation

words	Vectors
a	[0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
ability	[0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0]
able	[0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0]
⋮	⋮
zoo	[0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0]

- Sparse
- Each word is just a string or indices in the vocabulary list
- Hard to compute word relationships

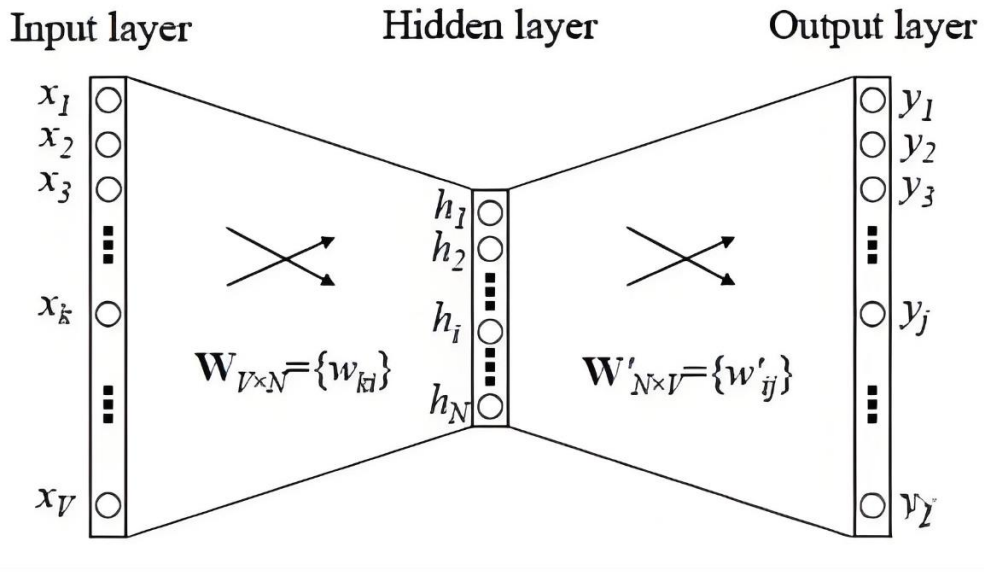
Word Embedding

- Each word = a vector
  - Similar words are “nearby in space”
  - The standard way to represent meaning in NLP

Word2vec

Distributed Representation—>Word2vec

- Alternative: we want to represent words as short (50-300 dimensional) & dense(real-valued) vectors
    - The basis for modern NLP systems



$$v_{cat} = \begin{pmatrix} -0.224 \\ 0.130 \\ -0.290 \\ \vdots \\ 0.276 \end{pmatrix}_{n\times 1} \qquad v_{dog} = \begin{pmatrix} -0.124 \\ 0.430 \\ -0.200 \\ \vdots \\ 0.329 \end{pmatrix}_{n\times 1}$$

$$v_{the} = \begin{pmatrix} 0.234 \\ 0.266 \\ 0.238 \\ \vdots \\ -0.199 \end{pmatrix}_{n\times 1} \qquad v_{language} = \begin{pmatrix} 0.194 \\ -0.447 \\ 0.780 \\ \vdots \\ 0.953 \end{pmatrix}_{n\times 1}$$

- Each word is represented by a low-dimensional (e.g., d = 300), real-valued vector
- Each coordinate/dimension of the vector doesn't have a particular interpretation

Word2vec

- Assumption: words under different context have the same vector
- Two word2vec algorithms:
  - Skip-gram:

    $$Likelihood:\ P(c|w;\ \theta)$$ Lec1

  - Continuous bag of words (CBOW):

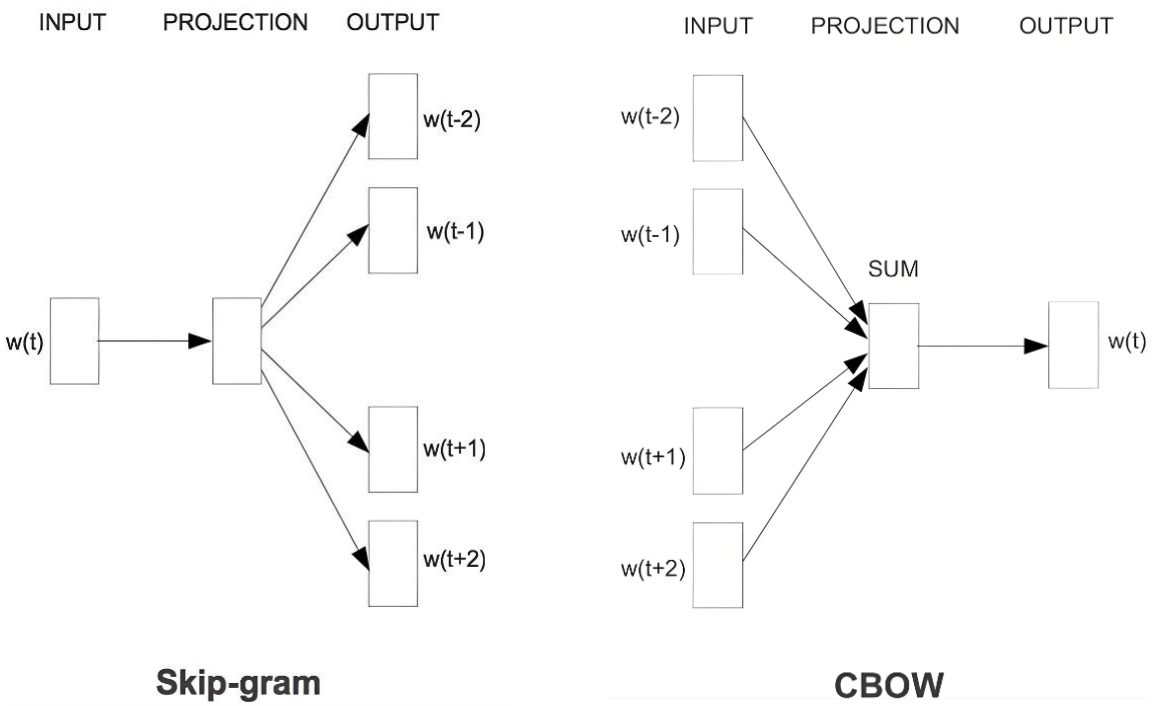
    $$Likelihood:\ P(w|c;\ \theta)$$

where

$\omega \in Text$

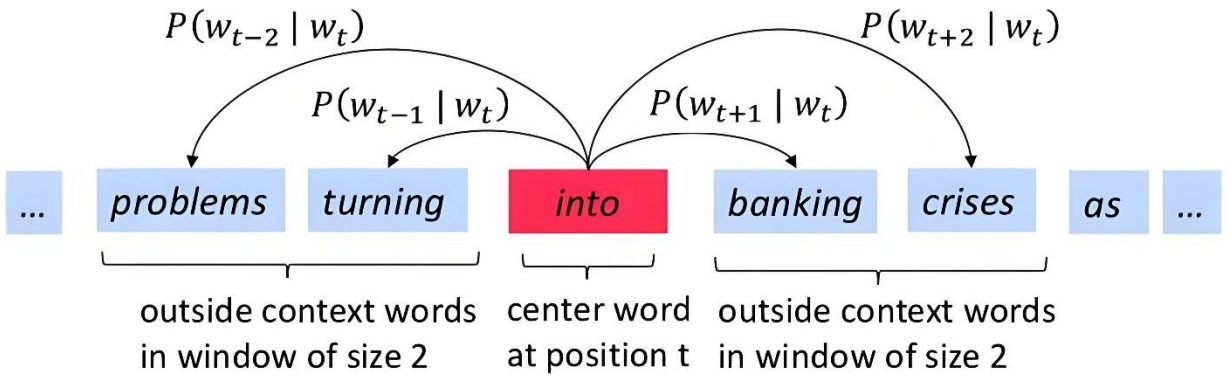
$c \in Context$

θ: all parameters to be optimized



Word2vec

- Skip-Gram example
  Context: a fixed window of size $2m$ (m = 2 in the example)



Convert the training data into:
(into, problems)
(into,turning)
(into,banking)
(into, crises)
(banking, turning)
(banking, into)
(banking, crises)
(banking, as)
...

Our goal is to find parameters that can maximize

$P(\text{problems} \mid \text{into}) \times P(\text{turning} \mid \text{into}) \times P(\text{banking} \mid \text{into}) \times P(\text{crises} \mid \text{into}) \times$ $P(\text{turning} \mid \text{banking}) \times P(\text{into} \mid \text{banking}) \times P(\text{crises} \mid \text{banking}) \times P(\text{as} \mid \text{banking}) \ldots$

Word2vec

- Objective

Usually, we optimize the model by minimizing the (average) negative log likelihood (NLL):

$$\underset{\theta}{\operatorname{argmin}}\, NLL(\theta) = \underset{\theta}{\operatorname{argmin}} -\log P(y|x;\ \theta)$$

Two word2vec algorithms:

- Skip-gram:

$$\underset{\theta}{\operatorname{argmax}} \prod_{\omega \in Text} \prod_{c \in Context} P(c|\omega;\ \theta) \Longrightarrow \mathrm{NLL}(\theta) = -\frac{1}{T}\sum_{t=1}^{T} \sum_{-m \le j \le m} logP\left(w_{t+j}|w_t;\ \theta\right)$$

- Continuous bag of words (CBOW):

$$\underset{\theta}{\operatorname{argmax}} \prod_{c \in Context} \prod_{\omega \in Text} P(\omega|c;\ \theta) \Longrightarrow \mathrm{NLL}(\theta) = -\frac{1}{2m} \sum_{-m \le j \le m} \sum_{t=1}^{T} logP\left(w_t|w_{t+j};\ \theta\right)$$

Maximum Likelihood Estimate (MLE)

where

m: context window size

T: the sentence length, $t + j \in [1, T]$

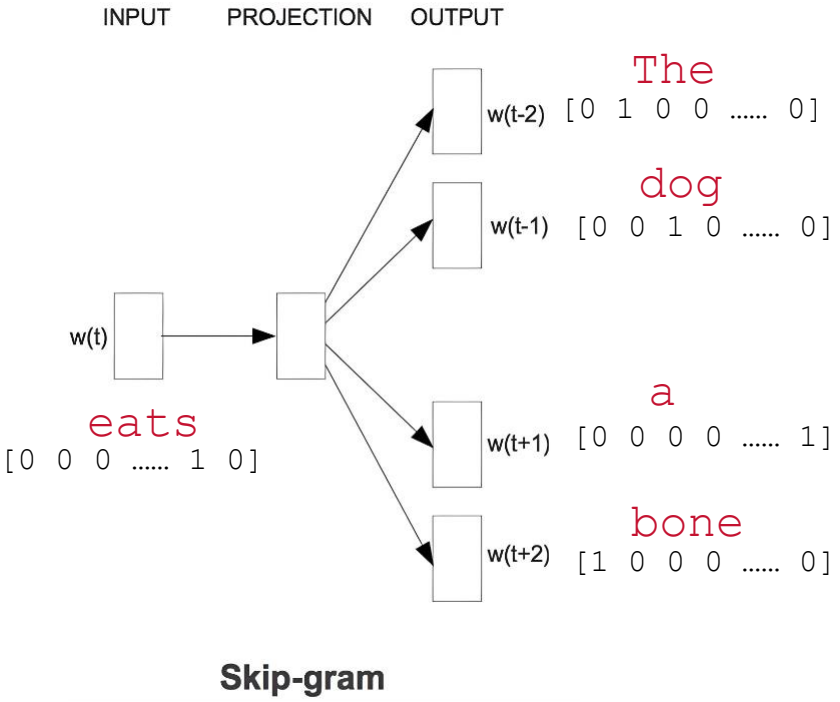
- The model parameters can be optimized with SGD,ADAM, etc.

Word2vec

Why NLL as an optimization objective?

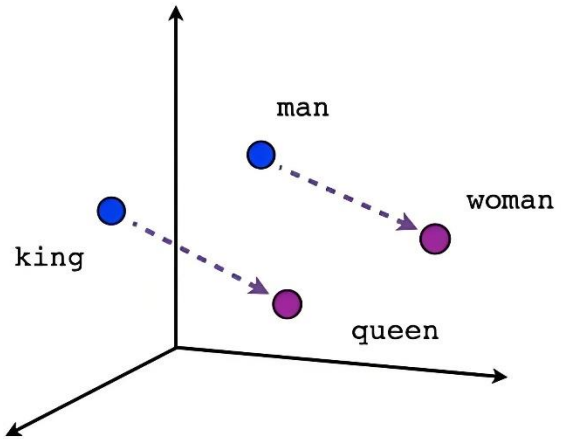
- NLL is suitable for probabilistic modeling tasks, especially when the model outputs a probability distribution.
- NLL goal is to optimize the matching degree between the predicted distribution and the true distribution.
- In classification, where the true labels are one-hot representation, minimizing NLL is equivalent to minimizing cross-entropy loss.

NLL = CE ?

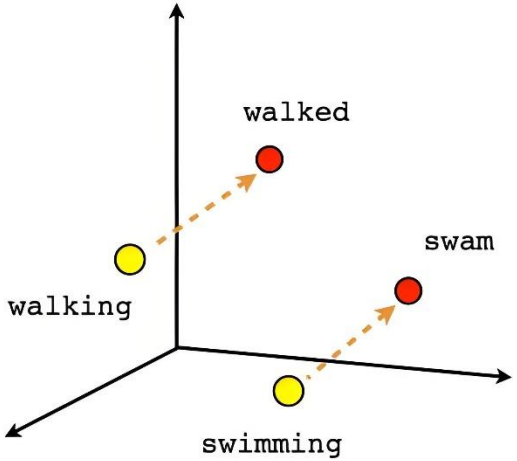


Word2vec

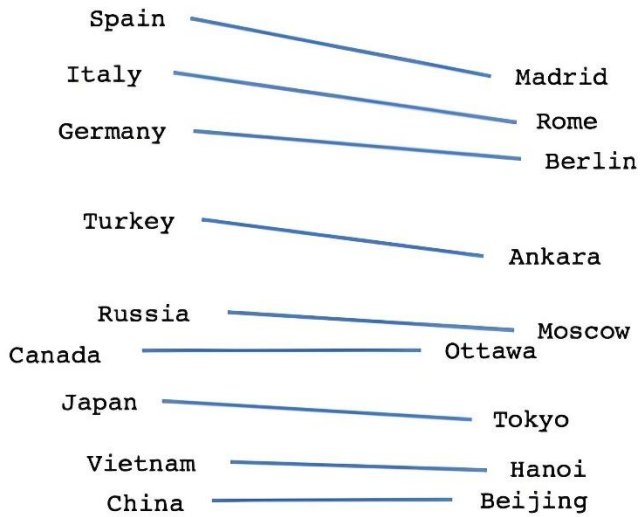
Word2vec: It has some nice properties



Male-Female



Verb tense



Country-Capital

$$v_{\text{man}} - v_{\text{woman}} \approx v_{\text{king}} - v_{\text{queen}}$$

$$v_{\text{Paris}} - v_{\text{France}} \approx v_{\text{Rome}} - v_{\text{Italy}}$$

Word analogy test: $a : a^* :: b : b^*$

$$b^* = \arg\max_{w \in V} \cos(e(w), e(a^*) - e(a) + e(b))$$

Word2vec

Word2vec: It has some problems

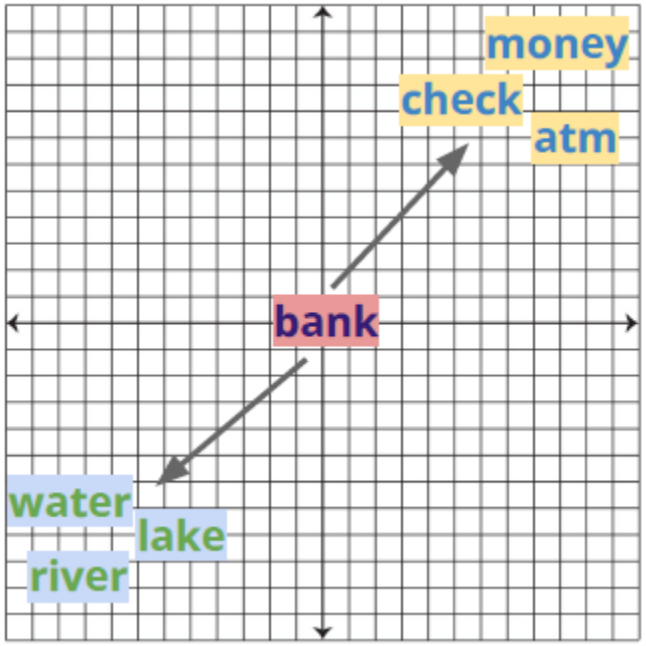
1. Static vectors cannot handle polysemy (multiple meanings of a word)

2. Relies on local context windows, lacking global semantic awareness

3. Insensitive to word order, ignoring grammatical structure, such as

"The cat chases the dog" ≈ *"The dog chases the cat. "*

4. Unable to handle out-of-vocabulary words



She has withdrawn the **money** from the **bank** account.

She stopped by the **river** **bank**.

Quiz

1. How to handle out-of-vocabulary words?

A. Character-level B. Word-level C. Subword-level

2. How is sensitive to word order, considering grammatical structure?

A. Inputting sequentially words into the model according to the order of word appearance

B. By teaching it grammatical rules

C. By using positional encoding to incorporate sequence information

Tokenizing Text

A tokenizer takes text and turns it into a sequence of discrete tokens.

A vocabulary is the list of all available tokens.

Let's tokenize: "A hippopotamus ate my homework."

Vocab Type	Example	Length
character-level	['A', ' ', 'h', 'i', 'p', 'p', 'o', 'p', 'o', 't', 'a', 'm', 'u', 's', ' ', 'a', 't', 'e', ' ', 'm', 'y', ' ', 'h', 'o', 'm', 'e', 'w', 'o', 'r', 'k', '.']	31
subword-level	['A', 'hip', '##pop', '##ota', '##mus', 'ate', 'my', 'homework', '.']	9
word-level	['A', 'hippopotamus', 'ate', 'my', 'homework', '.']	6

Tokenizing Text

A tokenizer takes text and turns it into a sequence of discrete tokens.

- **Subword tokenization**
  - High-frequency words remain as complete whole words.
  - Low-frequency words are split into meaningful subword units.
  - No out-of-vocabulary (OOV), thus preventing information loss.
  - The ability to learn relationships between prefixes, suffixes, and other morphemes.

Methods	Typical Models
BPE	GPT, LLaMA, BART, ChatGLM-6B
WordPiece	BERT, DistilBERT, MobileBERT
Unigram	AlBERT, T5, mBART, XLNet

Tokenizing Text

Byte Pair Encoding (BPE)

- A word segmentation algorithm
- Start with a vocabulary of characters
- Turn the most frequent n-gram to a new n-gram

Dictionary

Word	Freq
l o w	5
l o w e r	2
n e w e s t	6
w i d e s t	3

word2split →

Vocabulary

w, e, l, o, s, t, r, n, i, d

Tokenizing Text

Byte Pair Encoding (BPE)

- A word segmentation algorithm
- Start with a vocabulary of characters
- Turn the most frequent n-gram to a new n-gram

Dictionary

Word	Freq
l o w	5
l o w e r	2
n e w **e s** t	6
w i d **e s** t	3

word2split →

Vocabulary

w, e, l, o, s, t, r, n, i, d

pair2count ↓

a pair (e, s) with freq 9 → **es**

Tokenizing Text

Byte Pair Encoding (BPE)

- A word segmentation algorithm
- Start with a vocabulary of characters
- Turn the most frequent n-gram to a new n-gram

Dictionary

Word	Freq
l o w	5
l o w e r	2
n e w **e s** t	6
w i d **e s** t	3

word2split

Vocabulary

w, e, l, o, s, t, es, r, n, i, d

pair2count

a pair (e, s) with freq 9

es

update

Tokenizing Text

Byte Pair Encoding (BPE)

- A word segmentation algorithm
- Start with a vocabulary of characters
- Turn the most frequent n-gram to a new n-gram

Dictionary

Word	Freq
l o w	5
l o w e r	2
n e w **e s t**	6
w i d **e s t**	3

word2split

Vocabulary

w, e, l, o, s, t, es, est, r, n, i, d

pair2count

a pair (es, t) with freq 9

est

update

Tokenizing Text

Byte Pair Encoding (BPE)

- A word segmentation algorithm
- Start with a vocabulary of characters
- Turn the most frequent n-gram to a new n-gram

Dictionary

Word	Freq
l o w	5
l o w e r	2
n e w e s t	6
w i d e s t	3

word2split →

Vocabulary

w, e, l, o, s, t, es, est, lo, ow, low, r, n, i, d

pair2count ↓

pair frequency

↑ update

Tokenizing Text

WordPiece

Dictionary

Word	Freq
l o w	5
l o w e r	2
n e w e s t	6
w i d e s t	3

Vocabulary

l, o, w, e, r, n, w, s, t, i, d, lo,ow,es,low,est

pair2count

update

$$score = -\log(count(ab)/count(a)count(b))$$

Tokenizing Text

Unigram

Dictionary

Word	Freq
l o w	5
l o w e r	2
n e w e s t	6
w i d e s t	3

Vocabulary

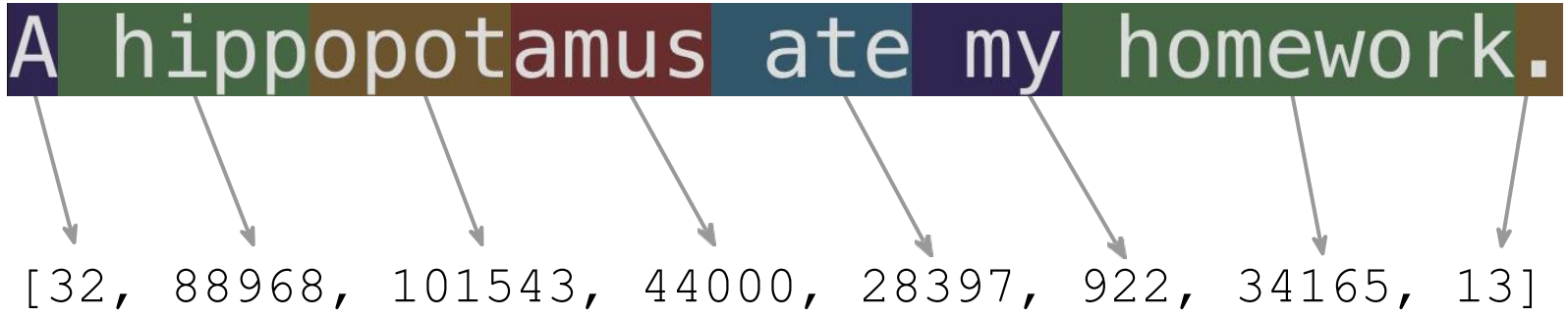
l, o, w, e, r, n, w, ..., low,lower,~~newest,widest~~

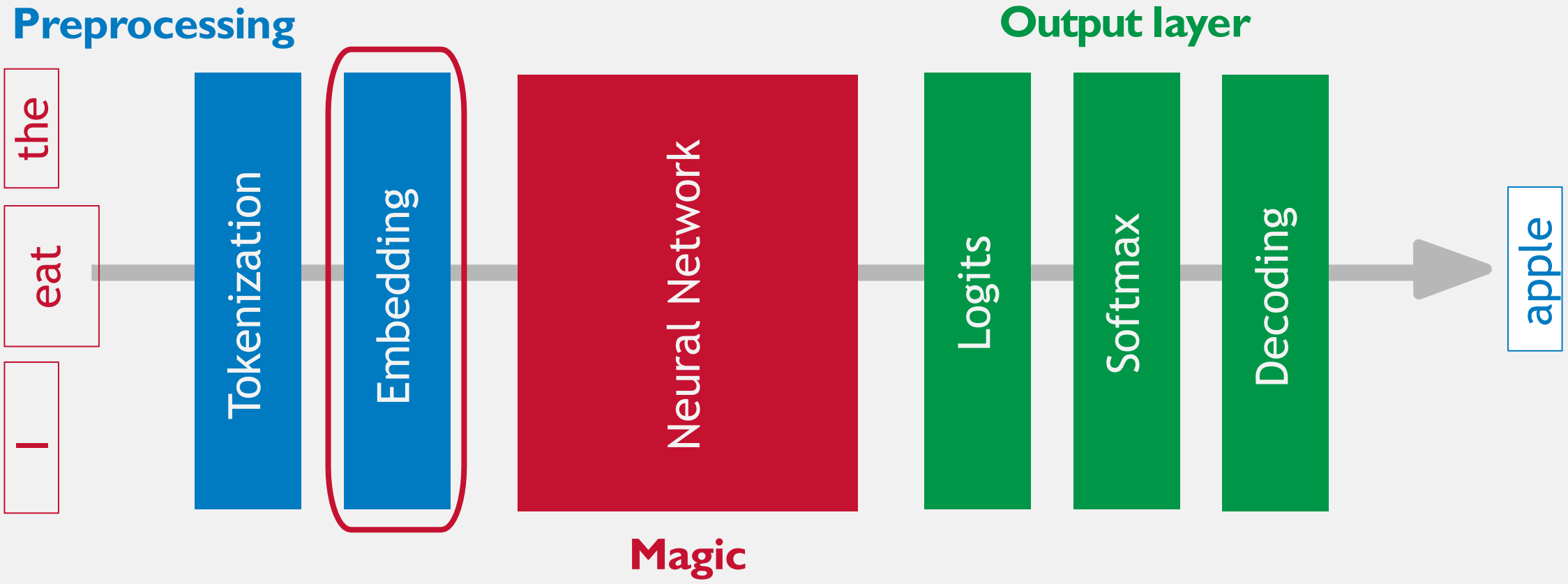
pair2count

update

$$score = -\log(count/total_count)$$

Tokenizing Text

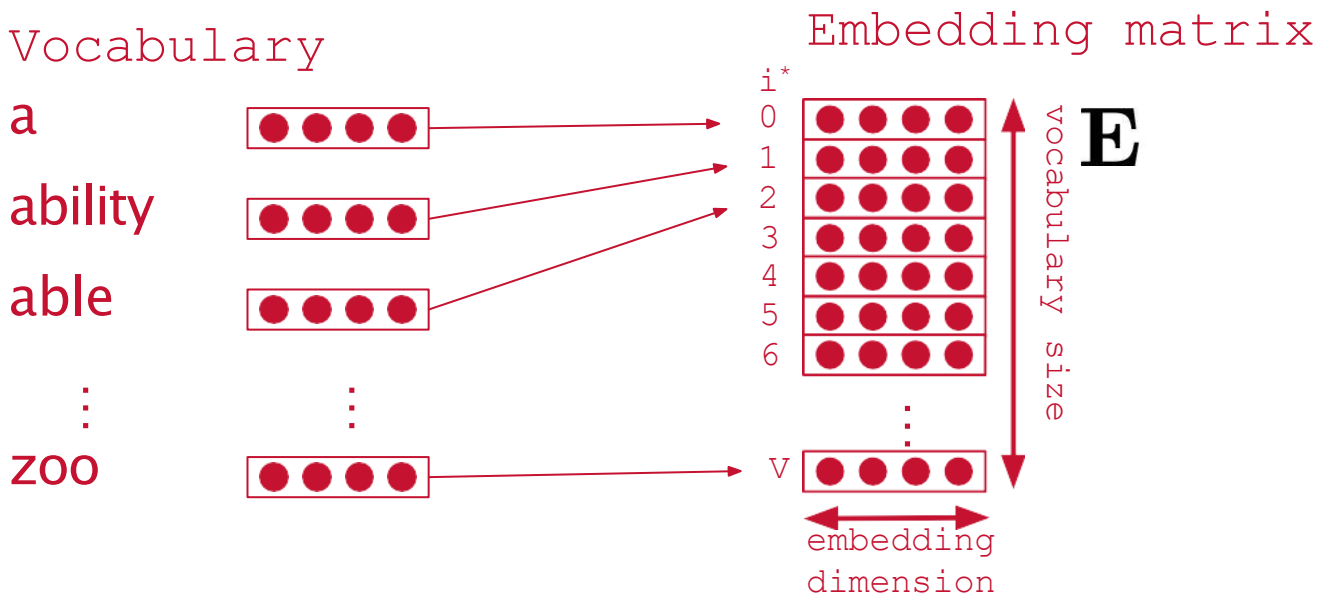



Preprocessing
I
eat
the
Tokenization
Embedding
Magic
Neural Network
Output layer
Logits
Softmax
Decoding
apple

Token Embedding

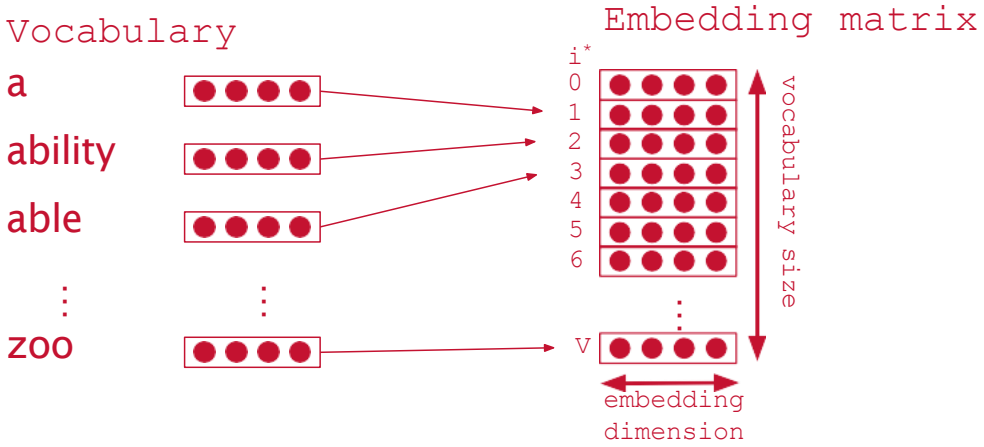
Turning Discrete Tokens into Continuous Vectors

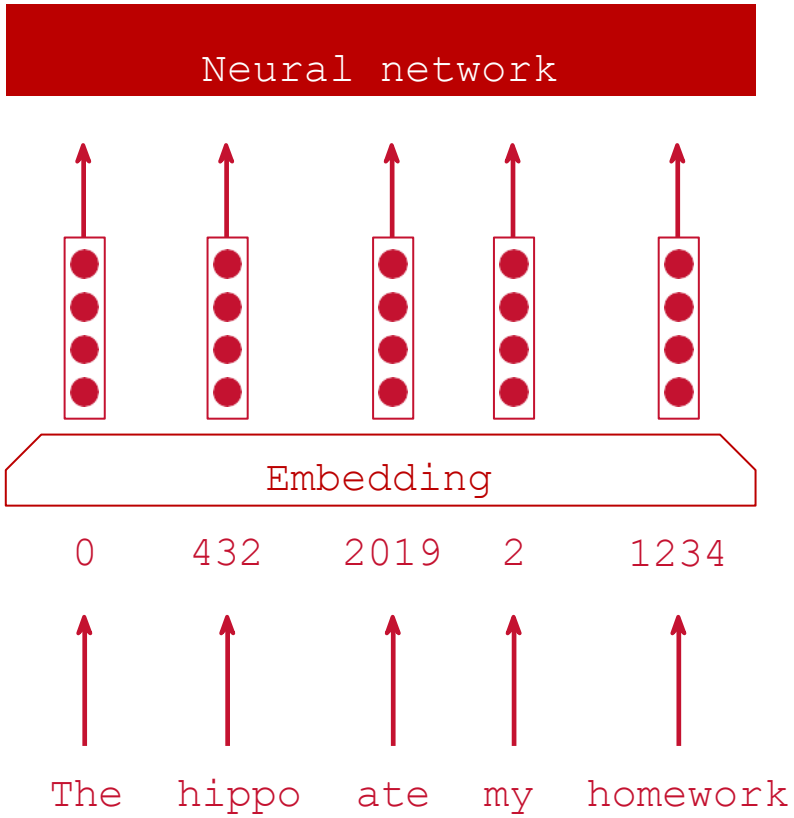
We build an embedding matrix which associates each token in the vocabulary with a vector embedding.



Embedding

- The LM takes as input the vectors of each token in the input sequence.

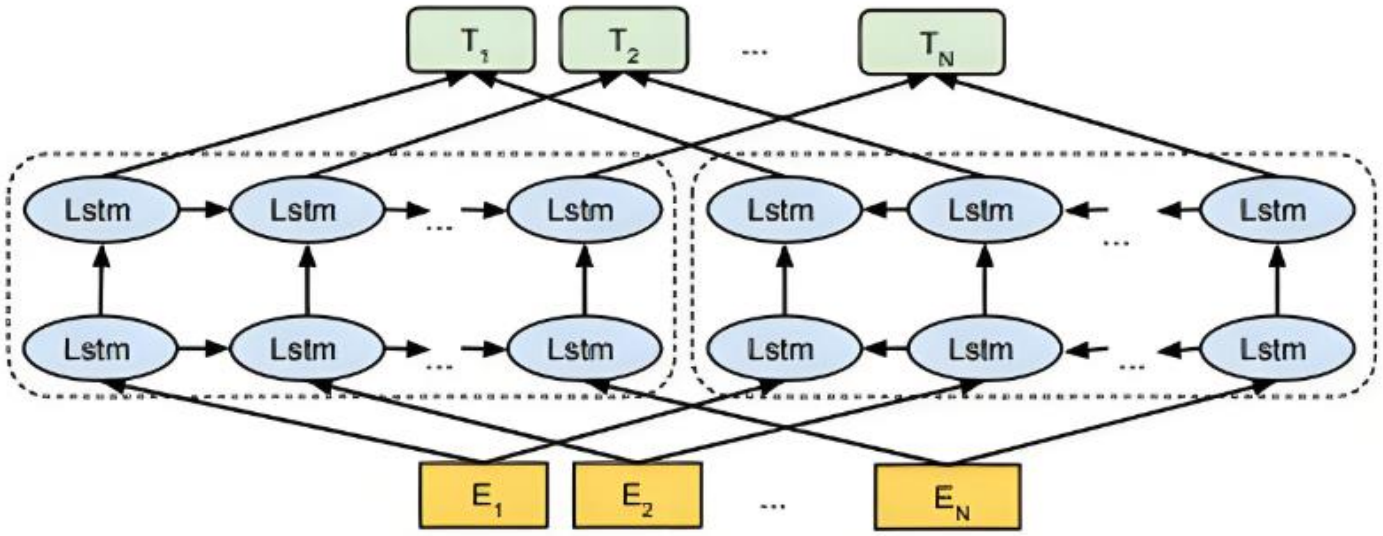




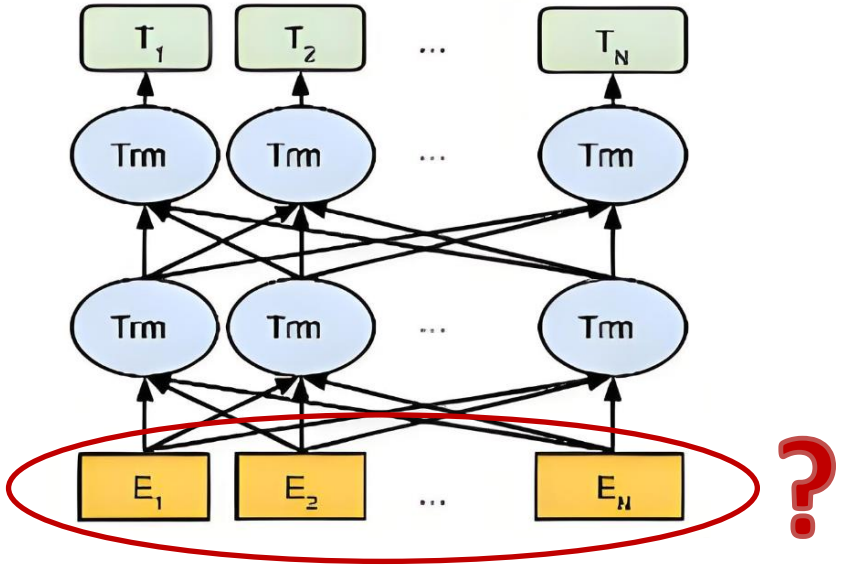
Embedding

How is sensitive to word order?

- Pretrained language models



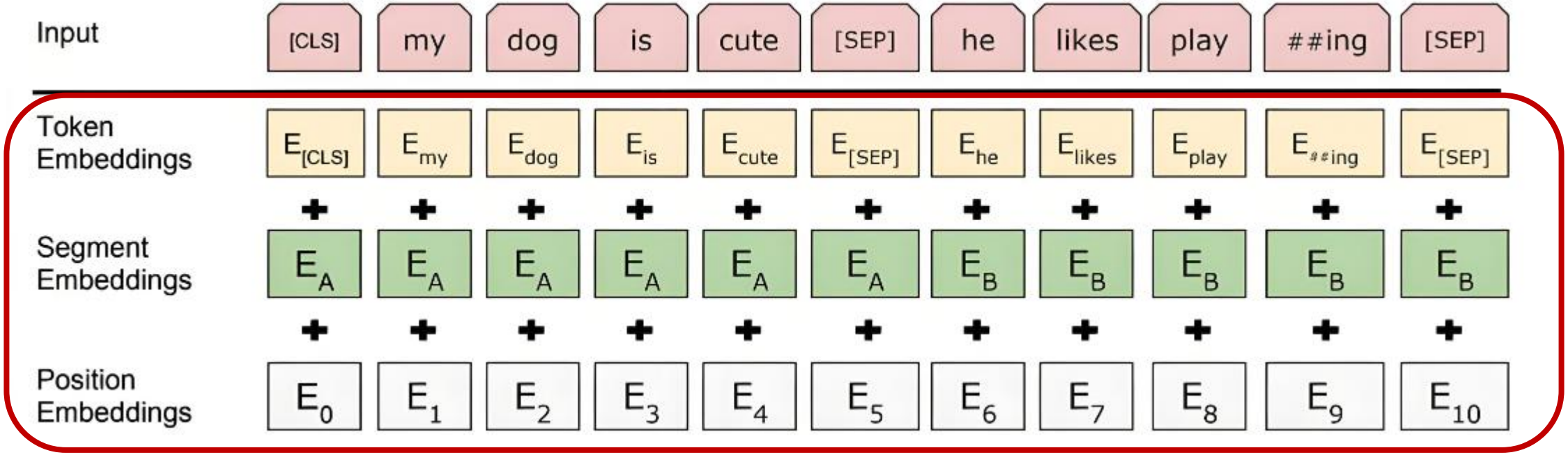
RNN-based model



Transformer-based model

Embedding

BERT:

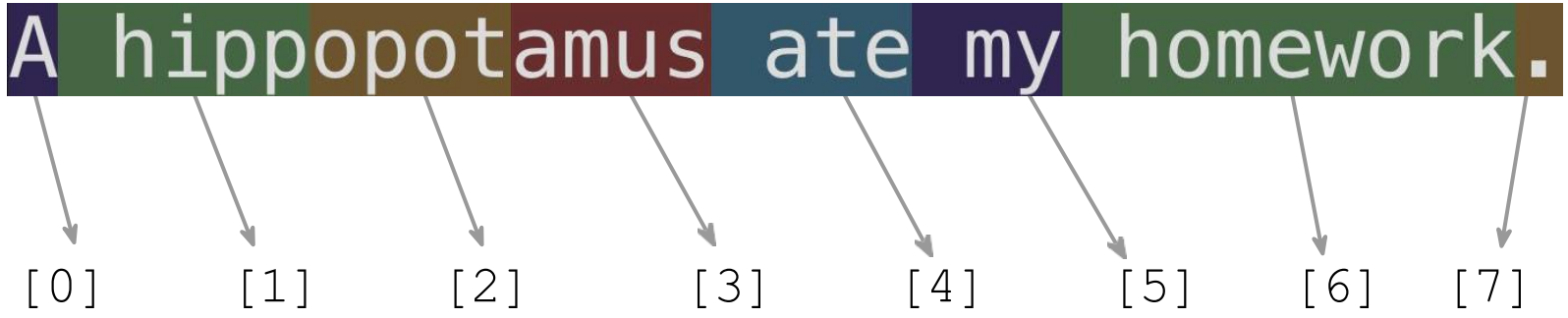


embedding block

Position Embedding

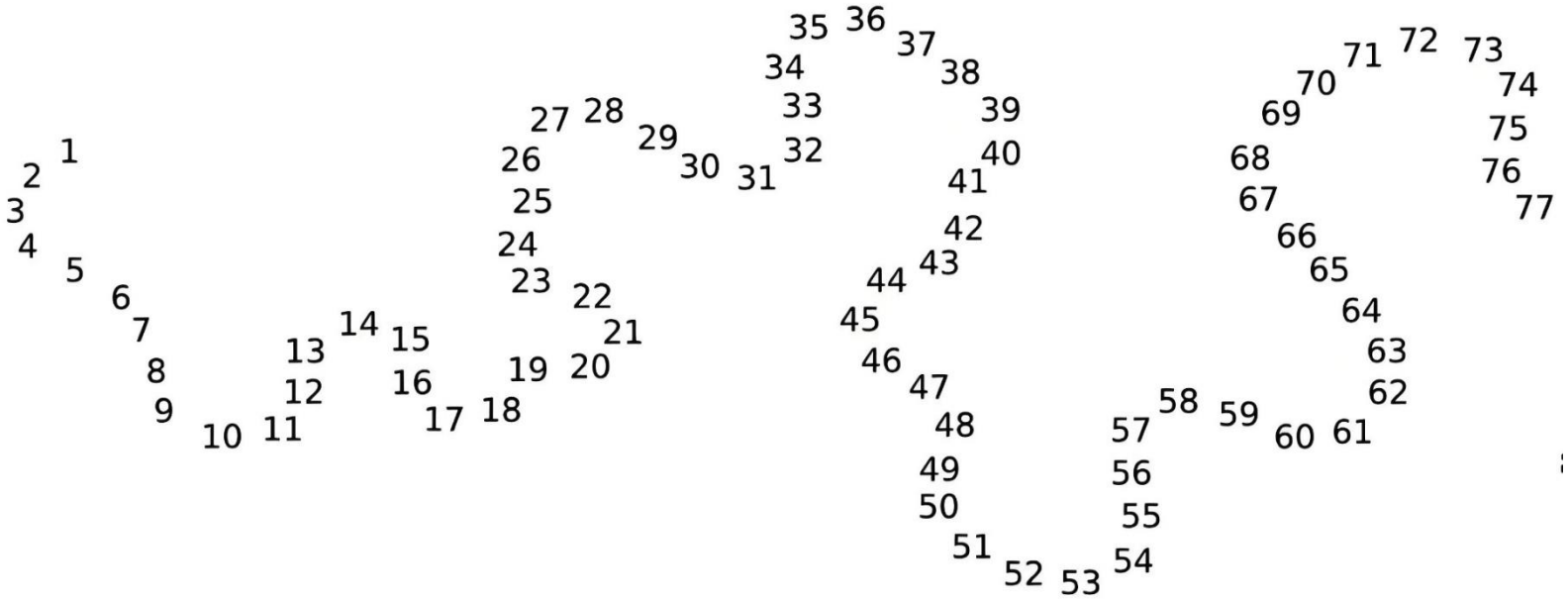
What is the position embedding?

- Inject positional information of words in the sequence into the model.



Position Embedding

Absolute Positional Encoding



Position Embedding

Sinusoidal Positional Encoding (SPE)

$$PE_{(pos,2i)} = \sin\left(pos/10000^{2i/d}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(pos/10000^{2i/d}\right)$$

Where pos: position
d: the embedding dimension
i: the ith dimension

The values of the position encoding are fixed between [-1, 1] .

Why use a Sinusoidal Function?

Position Embedding

What are the properties of Sinusoidal position embedding?

$$PE(t, 2i) = sin(t * w_{2i})$$
$$PE(t, 2i+1) = cos(t * w_{2i})$$

where $w_{2i} = 1/10000^{2i/d}$ and $t = pos$

For a fixed position distance k,

$PE(t+k)$ can be expressed as a linear function of $PE(t)$

$$\begin{aligned} PE(t+k, 2i) &= sin(t * w_{2i} + k * w_{2i}) \\ &= sin(t * w_{2i}) cos(k * w_{2i}) + cos(t * w_{2i}) sin(k * w_{2i}) \\ &= PE(t, 2i) \underline{PE(k, 2i+1)} + PE(t, 2i+1) \underline{PE(k, 2i)} \\ &= PE(t, 2i) \underline{u} + PE(t, 2i+1) \underline{v} \end{aligned}$$

$$PE(t+k, 2i+1) = PE(t, 2i+1)\, u - PE(t, 2i)\, v$$

Position Embedding

What are the properties of sinusoidal position embedding?

$$PE(t, 2i) = sin(t * w_{2i})$$
$$PE(t, 2i+1) = cos(t * w_{2i})$$

where $w_{2i} = 1/10000^{2i/d}$ and $t = pos$

The inner product of two position vectors only depends on the relative position k.

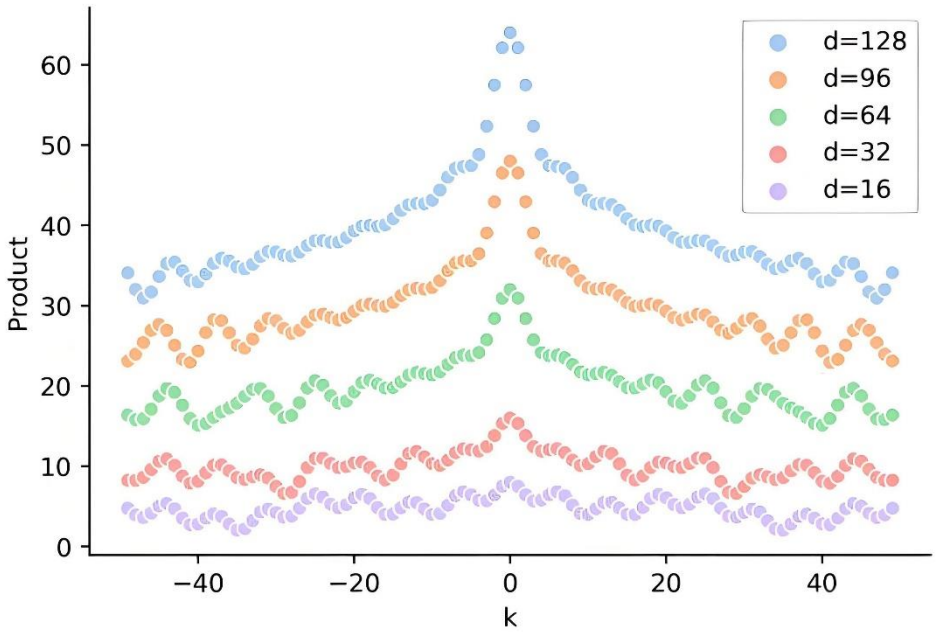
$$PE(t)\,PE(t+k) = \sum_{i=0}^{d/2-1} PE(t, 2i)\,PE(t+k, 2i) + \sum_{i=0}^{d/2-1} PE(t, 2i+1)\,PE(t+k, 2i+1)$$
$$= \sum_{i=0}^{d/2-1} sin(t * w_{2i})\,sin[(t+k) * w_{2i}] + \sum_{i=0}^{d/2-1} cos(t * w_{2i})\,cos[(t+k) * w_{2i}]$$
$$= \sum_{i=0}^{d/2-1} cos(k * w_{2i})$$

$$PE(t+k)PE(t) = PE(t)PE(t-k)$$

Sinusoidal position embedding is symmetric!

Position Embedding

What are the properties of Sinusoidal position embedding?



As k increases, the inner product results of position vectors gradually decrease, which means there is a long-term decay.

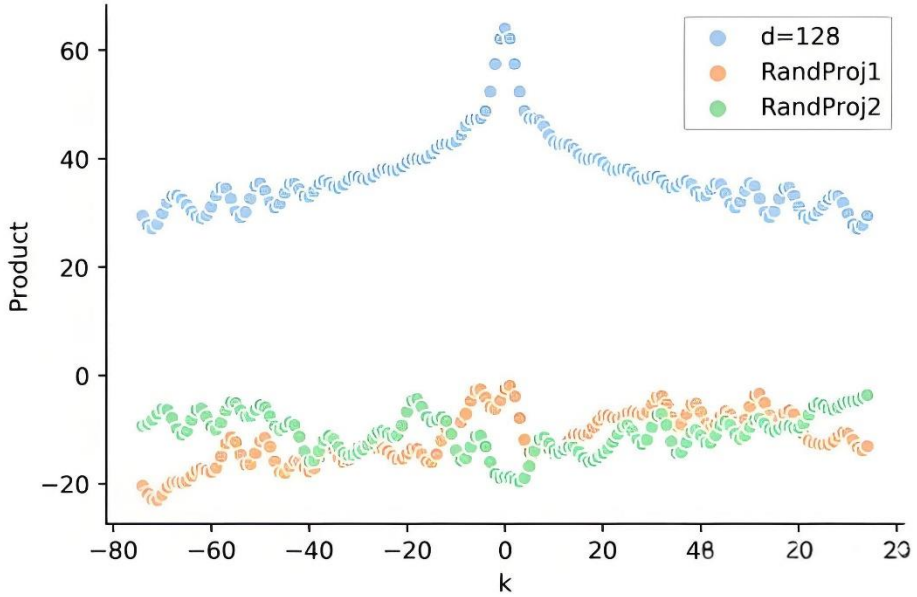
Position Embedding

What are the properties of Sinusoidal position embedding?

In Attention computation, it also needs to be multiplied by the attention weight W,

$$PE_t^T \boldsymbol{W}_q^T \boldsymbol{W}_k PE_{t+k}$$

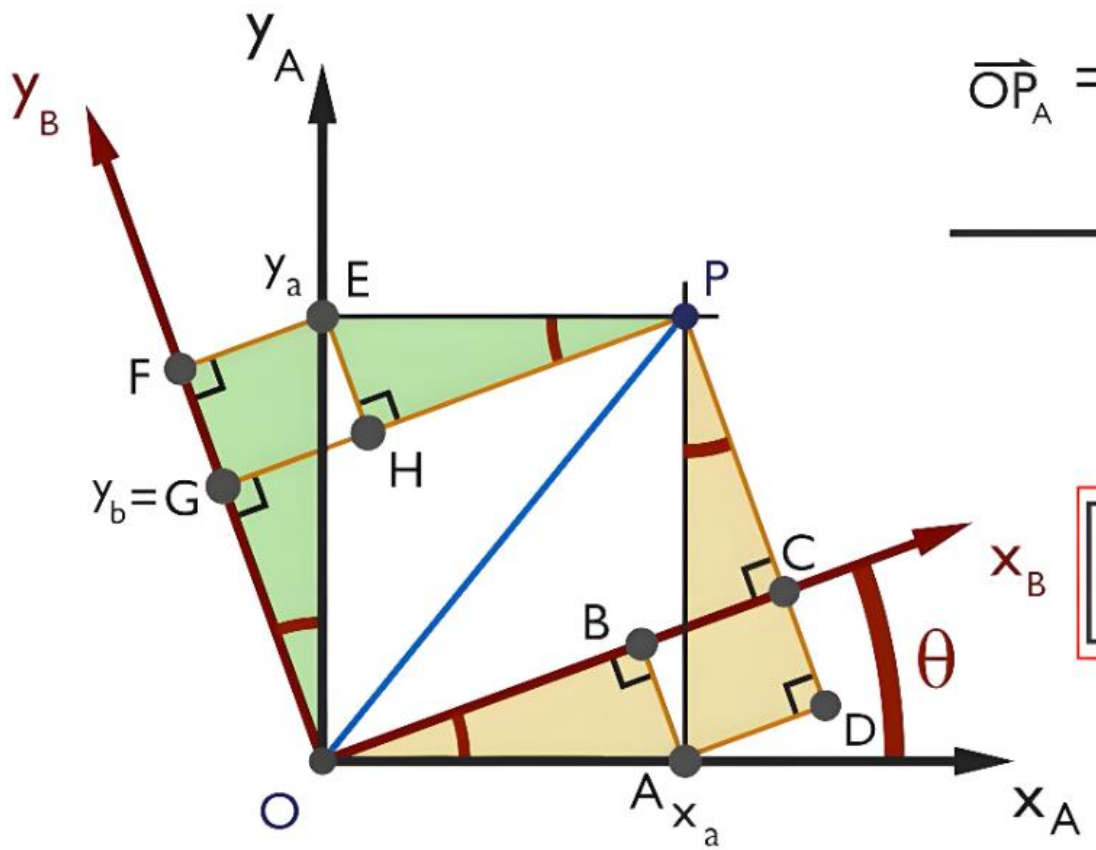
Inner product result can no longer reflect the relative distance.



Position Embedding

Rotary Position Embedding (RoPE)

- What is rotation matrix?



$$\overrightarrow{OP}_A = \begin{bmatrix} x_a \\ y_a \end{bmatrix} \quad \overrightarrow{OP}_B = \begin{bmatrix} x_b \\ y_b \end{bmatrix}_A = ?$$

$$x_b = x_a \cos\theta + y_a \sin\theta$$

$$y_b = -x_a \sin\theta + y_a \cos\theta$$

$$\boxed{\begin{bmatrix} x_b \\ y_b \end{bmatrix}} = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix} \boxed{\begin{bmatrix} x_a \\ y_a \end{bmatrix}}$$

$$\overrightarrow{OP}_B = \underline{\underline{T}}\ \overrightarrow{OP}_A$$

Position Embedding

Rotary Position Embedding (RoPE)

The self-attention first incorporates position information to the word embeddings and transforms them into queries, keys, and value representations.

$$\boldsymbol{q}_m = f_q(\boldsymbol{x}_m, m)$$
$$\boldsymbol{k}_n = f_k(\boldsymbol{x}_n, n)$$
$$\boldsymbol{v}_n = f_v(\boldsymbol{x}_n, n),$$

where

-x_m and x_n are the word embeddings of the $m^{th}$and n^{th} token, without position information.

-f_q, f_k and f_v are the networks for getting the query, key, and value representations, respectively.

- q_m, k_n and V_n incorporate the $m^{th}$and n^{th}positions through f_q, f_k and f_v.

Position Embedding

Rotary Position Embedding (RoPE)

we hope that the inner product encodes position information only in the relative form:

$$\underbrace{\langle f_q(\boldsymbol{x}_m, m), f_k(\boldsymbol{x}_n, n)\rangle}_{\text{inner product}} = \underbrace{g(\boldsymbol{x}_m, \boldsymbol{x}_n, m-n)}_{\text{A function } g(\cdot)\text{, which takes only the word embeddings } x_m \text{ and } x_n \text{ , and their relative position } m-n \text{ as input variables}}.$$

Position Embedding

Rotary Position Embedding (RoPE)

- A simple case with a dimension d = 2.

$$\boxed{e^{im\theta} = \cos(m\theta) + isin(m\theta)}$$

$$f_q(\boldsymbol{x}_m, m) = (\boldsymbol{W}_q \boldsymbol{x}_m) e^{im\theta}$$
$$f_k(\boldsymbol{x}_n, n) = (\boldsymbol{W}_k \boldsymbol{x}_n) e^{in\theta}$$

where
-W_q and W_k are the attention weights for query and key.
-$e^{im\theta}$: the position information.

Position Embedding

Rotary Position Embedding (RoPE)

- A simple case with a dimension d = 2.

$$f_q(x_m, m) = \underline{(W_q x_m)} e^{im\theta} = \underline{q_m} e^{im\theta}$$

$$q_m e^{im\theta} = (q_m^{(1)} + i q_m^{(2)}) * (\cos(m\theta) + i \sin(m\theta))$$
$$= (q_m^{(1)} cos(m\theta) - q_m^{(2)} \sin(m\theta)) + i(q_m^{(2)} \cos(m\theta) + q_m^{(1)} \sin(m\theta))$$

$$f_q(\boldsymbol{x}_m, m) = \boxed{\begin{pmatrix} \cos m\theta & -\sin m\theta) \\ \sin m\theta & \cos m\theta \end{pmatrix}} \begin{pmatrix} q_m^{(1)} \\ q_m^{(2)} \end{pmatrix}$$

$$f_k(\boldsymbol{x}_m, m) = \boxed{\begin{pmatrix} \cos m\theta & -\sin m\theta) \\ \sin m\theta & \cos m\theta \end{pmatrix}} \begin{pmatrix} k_m^{(1)} \\ k_m^{(2)} \end{pmatrix}$$

Position Embedding

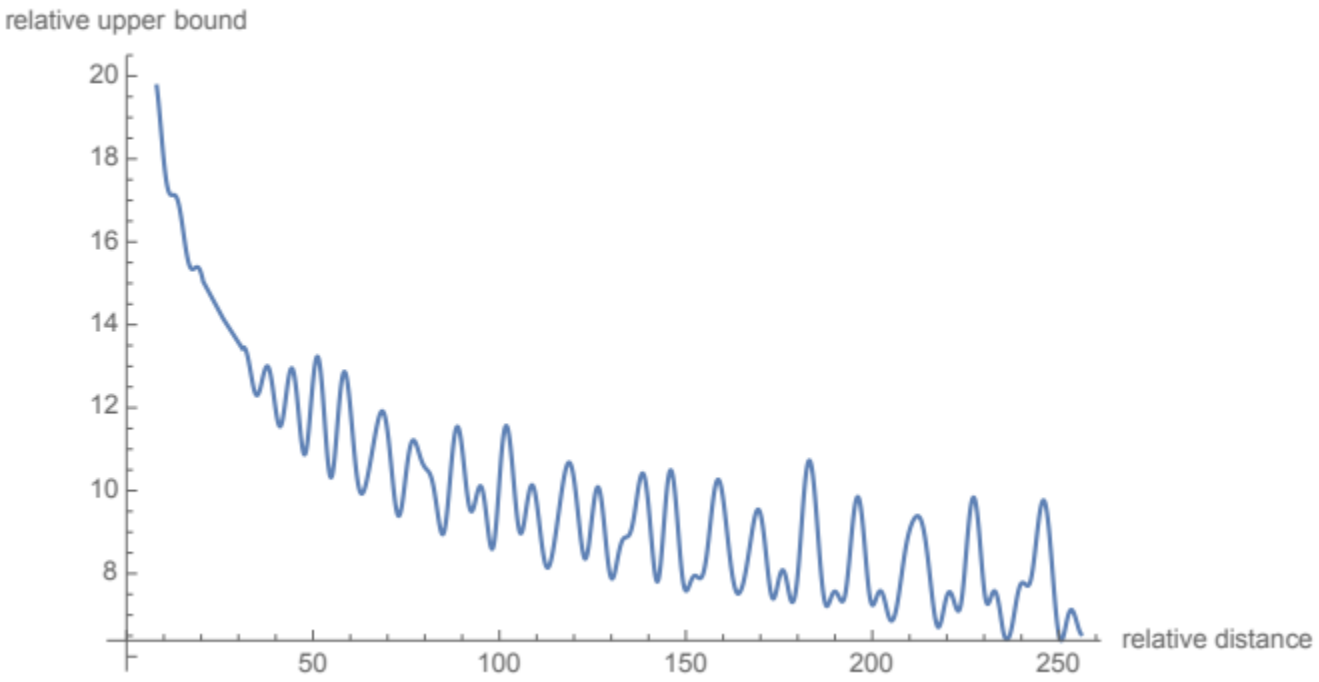
Rotary Position Embedding (RoPE)

$$
\begin{aligned}
&< f_q(x_m, m), f_k(x_n, n) >= \\
&q_m^{(1)} k_n^{(1)} (\cos(m\theta)\cos(n\theta) + \sin(m\theta)\sin(n\theta)) \\
&+ q_m^{(1)} k_n^{(2)} (-\cos(m\theta)\sin(n\theta) + \sin(m\theta)\cos(n\theta)) \\
&+ q_m^{(2)} k_n^{(1)} (-\sin(m\theta)\cos(n\theta) + \cos(m\theta)\sin(n\theta)) \\
&+ q_m^{(2)} k_n^{(2)} (\sin(m\theta)\sin(n\theta) + \cos(m\theta)\cos(n\theta)) \\
&= q_m^{(1)} k_n^{(1)} \cos((m-n)\theta) \\
&+ q_m^{(1)} k_n^{(2)} \sin((m-n)\theta) \\
&- q_m^{(2)} k_n^{(1)} \sin((m-n)\theta) \\
&+ q_m^{(2)} k_n^{(2)} \cos((m-n)\theta) \\
&= (q_m^{(1)} k_n^{(1)} + q_m^{(2)} k_n^{(2)}) \cos((m-n)\theta) + (q_m^{(1)} k_n^{(2)} - q_m^{(2)} k_n^{(1)}) \sin((m-n)\theta) \\
&= (q_m^{(1)} k_n^{(1)} + q_m^{(2)} k_n^{(2)}) \cos((m-n)\theta) - (q_m^{(2)} k_n^{(1)} - q_m^{(1)} k_n^{(2)}) \sin((m-n)\theta) \\
&= g(x_m, x_n, m-n)
\end{aligned}
$$

Position Embedding

Rotary Position Embedding (RoPE)

- Long-term decay of RoPE.



Thank you!

SUSTech Southern University of Science and Technology